

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4305

COURSE OUTCOME COVERED

CO 3: Implement dynamic web applications by utilizing DOM-based client-side scripting and employing Java Servlets for server-side request processing and response handling. (L3)

Assessment Tool : Scenario-Based Assessment

Weightage: 40%

QUESTIONS:

Total Marks: 30

Question 1

Suppose that a college wants to develop an online student registration system where students submit their details through a web form, and the data is processed using a Java Servlet. Explain the architecture of a servlet-based application. Design and implement a servlet to handle form data using both GET and POST methods. Describe the servlet life cycle (init, service, destroy) and justify which method is more appropriate for handling sensitive data.

Question 2

Suppose that a website requires users to log in and maintain their session across multiple pages after authentication. Design a session management system using HttpSession and Cookies in Servlets. Explain how session tracking works in web applications and compare session-based tracking with cookies.

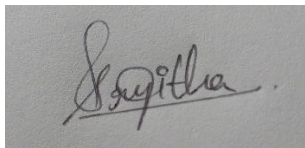
Question 3

Suppose that an e-commerce platform needs to store and retrieve product details from a database using a Java-based web application.

Design a JDBC-based application to connect to a database, insert product data, and retrieve it using SQL queries. Explain the JDBC architecture and describe the steps involved in database connectivity along with proper exception handling.

Code of Conduct:

I S.SRIJITHA(192471024)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

QUESTION 1 – Servlet-Based Student Registration System

Scenario

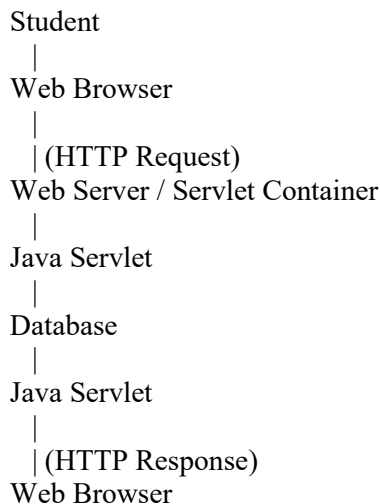
A college wants to develop an online student registration system where students enter their details through a web form. The submitted data is processed by a Java Servlet.

1. Architecture of a Servlet-Based Application

A servlet-based web application follows a **client-server architecture**. The main components are:

1. **Web Browser:** The student enters registration details and sends an HTTP request.
2. **Web Server:** The server receives the request and forwards it to the servlet container.
3. **Servlet Container:** It manages the servlet, handles requests, and controls the servlet life cycle.
4. **Java Servlet:** The servlet processes the form data using doGet() or doPost().
5. **Database:** Student information can be stored and retrieved from a database.
6. **HTTP Response:** The servlet sends the processed result back to the student's browser.

Architecture Flow



2. HTML Form for Student Registration

register.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Student Registration</title>
</head>
<body>
<h2>Student Registration Form</h2>
<form action="Register Servlet" method="post"
  Name:
  <input type="text" name="name"><br><br>
  Roll Number:
  <input type="text" name="roll no"><br><br>
  Email:
  <input type="email" name="email"><br><br>
  Course:
  <input type="text" name="course"><br><br>
  <input type="submit" value="Register"
</form>
</body>
</html>
```

The form uses the **POST method**, which is suitable for submitting registration information.

3. Java Servlet Using GET and POST

RegisterServlet.java

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class RegisterServlet extends HttpServlet {
    public void init() throws ServletException {
        System.out.println("Servlet Initialized");
    }
    protected void doGet (Http Servlet Request request,
        Http Servlet Response response)
        throws Servlet Exception, IOException {
        process Request(request, response, "GET");
    }
    protected void doPost(Http Servlet Request request,
        Http Servlet Response response)
        throws Servlet Exception, IOException {
        process Request(request, response, "POST");
    }
    private void process Request(Http Servlet Request request,
        Http Servlet Response response,
        String method)
        throws IOException {
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        String name = request.getParameter("name");
        String rollno = request.getParameter("rollno");
        String email = request.getParameter("email");
        String course = request.getParameter("course");
        out.println("<html><body>");
        out.println("<h2>Student Registration Successful</h2>");
        out.println("<p>Request Method: " + method + "</p>");
        out.println("<p>Name: " + name + "</p>");
        out.println("<p>Roll Number: " + rollno + "</p>");
        out.println("<p>Email: " + email + "</p>");
        out.println("<p>Course: " + course + "</p>");
        out.println("</body></html>");
    }
    public void destroy() {
        System.out.println("Servlet Destroyed");
    }
}
```

Optional web.xml

```
<xml version="1.0" encoding="UTF-8"?>
<web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
        http://xmlns.jcp.org/xml/ns/javaee/web-app_3_1.xsd"
    version="3.1">
    <servlet>
        <servlet-name>RegisterServlet</servlet-name>
        <servlet-class>RegisterServlet</servlet-class>
    </servlet>
    <servlet-mapping>
        <servlet-name>RegisterServlet</servlet-name>
        <url-pattern>/RegisterServlet</url-pattern>
    </servlet-mapping>
</web-app>
```

4. Servlet Life Cycle

The servlet has three important life-cycle methods:

init()

The init() method is called **only once** when the servlet is created by the servlet container.

```
public void init() {  
    System.out.println("Servlet Initialized");  
}
```

It can be used for initialization tasks such as creating resources or loading configuration.

service()

The service() method is called for **every client request**. In HttpServlet, it internally directs the request to methods such as doGet() and doPost().

Client Request



```
service()  
/  \  
doGet() doPost()
```

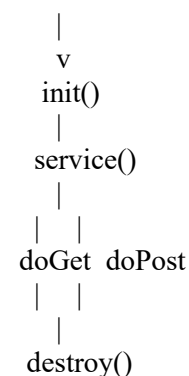
destroy()

The destroy() method is called **once before the servlet is removed** from the server.

```
public void destroy() {  
    System.out.println("Servlet Destroyed");  
}
```

Life-Cycle Flow

Servlet Loaded



5. GET vs POST for Sensitive Data

Feature	GET	POST
Data location	URL	Request body
Visibility	Visible in URL	Not displayed in URL
Data size	Limited	Larger data can be submitted
Security	Less suitable for sensitive data	More appropriate
Example	Search request	Login/registration

POST is more appropriate for sensitive data such as passwords, personal information, and registration details because the submitted values are placed in the HTTP request body rather than directly in the URL.

However, **POST alone does not encrypt data**. HTTPS should also be used to protect sensitive information during transmission.

QUESTION 2 – Session Management Using Http Session and Cookies

Scenario

A website requires users to log in and remain authenticated while moving between different pages.

1. Session Management

HTTP is a **stateless protocol**, meaning that the server does not automatically remember a user between separate requests.

For example:

Login Page

|

Login Servlet

|

Authentication Successful

|

Create Session

|

Home Page

|

Profile Page

A session management mechanism allows the application to identify the same user across multiple requests.

2. Using HttpSession

LoginServlet.java

```
import java.io.*;
```

```
import javax.servlet.*;
```

```
import javax.servlet.http.*;
```

```
public class LoginServlet extends HttpServlet {

    protected void doPost(HttpServletRequest request,
                           HttpServletResponse response)
        throws ServletException, IOException {
        response.setContentType("text/html");
        String username = request.getParameter("username");
        String password = request.getParameter("password");
        if (username.equals("admin") &&
            password.equals("1234")) {
            HttpSession session = request.getSession();
            session.setAttribute("username", username);
            response.sendRedirect("HomeServlet");
        } else {
            PrintWriter out = response.getWriter();
            out.println("<h3>Invalid Username or Password</h3>");
        }
    }
}
```

HomeServlet.java

```
import java.io.*;
```

```
import javax.servlet.*;
```

```
import javax.servlet.http.*;
```

```
public class HomeServlet extends HttpServlet {

    protected void doGet(HttpServletRequest request,
                           HttpServletResponse response)
        throws ServletException, IOException {

        response.setContentType("text/html");

        HttpSession session = request.getSession(false);
```

```

        PrintWriter out = response.getWriter();

        if (session != null &&
            session.getAttribute("username") != null) {

            String username =
                (String) session.getAttribute("username");

            out.println("<h2>Welcome " + username + "</h2>");
            out.println("<p>You are successfully logged in.</p>");

        } else {

            out.println("<h3>Please login first.</h3>");

        }
    }
}

```

3. Using Cookies

A cookie stores a small amount of information in the user's browser.

Creating a Cookie

```
Cookie cookie = new Cookie("username", username);
```

```
cookie.setMaxAge(60 * 60);
```

```
response.addCookie(cookie);
```

Reading a Cookie

```
Cookie[] cookies = request.getCookies();
```

```

if (cookies != null) {

    for (Cookie cookie : cookies) {

        if (cookie.getName().equals("username")) {

            String username = cookie.getValue();

            System.out.println("User: " + username);

        }
    }
}

```

4. How Session Tracking Works

Session tracking can be performed using different techniques such as:

- Cookies
- HttpSession
- URL rewriting
- Hidden form fields

With HttpSession, the server creates a unique **session ID** for the user. The browser normally stores the session ID in a cookie called JSESSIONID.

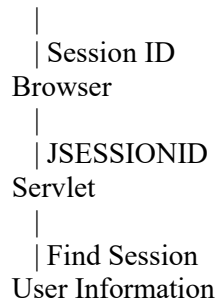
Browser

|
| Login Request

Servlet

|
| Create HttpSession

Server



5. Session vs Cookies

Feature	Http Session	Cookies
Storage	Mainly server-side	Client/browser-side
Security	Generally safer for application data	More exposed to client
Data capacity	Larger	Small
Lifetime	Usually controlled by session timeout	Can have expiration time
Main purpose	Maintain user session	Store small pieces of information
Example	Logged-in user information	Preferences or identifiers

Http Session is generally preferred for maintaining authenticated user information because the actual session data is maintained on the server. Cookies can be used to store small client-side values or to carry the session identifier.

QUESTION 3 – JDBC-Based E-Commerce Application

Scenario

An e-commerce application needs to store and retrieve product information from a database using Java.

1. JDBC

JDBC stands for Java Database Connectivity.

It is a Java API that allows a Java application to communicate with relational databases such as MySQL, Oracle, and PostgreSQL.

The application can use JDBC to:

- Establish a database connection.
- Insert product information.
- Execute SQL queries.
- Retrieve product information.
- Update and delete records.

2. JDBC Architecture

Java Application

JDBC API

JDBC Driver Manager

JDBC Driver

Database

Components

1. Java Application:

The Java program that needs to access the database.

2. JDBC API:

Provides classes and interfaces such as Connection, Statement, PreparedStatement, and ResultSet.

3. Driver Manager:

Manages JDBC database drivers and helps establish database connections.

4. JDBC Driver:

Provides communication between Java and the specific database.

5. Database:

Stores product information.

3. Database Table

For example, create a MySQL database and table:

```
CREATE DATABASE ecommerce;
```

```
USE ecommerce;
```

```
CREATE TABLE products (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    name VARCHAR(100),  
    price DOUBLE,  
    quantity INT  
);
```

4. JDBC Steps

The main steps involved in JDBC database connectivity are:

Step 1 – Load the JDBC Driver

```
Class.forName("com.mysql.cj.jdbc.Driver");
```

Step 2 – Establish Connection

```
Connection con = DriverManager.getConnection(  
    "jdbc:mysql://localhost:3306/ecommerce",  
    "root",  
    "1234"  
);
```

Step 3 – Create SQL Statement

```
PreparedStatement ps =  
    con.prepareStatement(  
        "INSERT INTO products(name, price, quantity) VALUES (?, ?, ?)"  
    );
```

Step 4 – Execute SQL Query

```
ps.executeUpdate();
```

Step 5 – Process the Result

For a SELECT query:

```
ResultSet rs = ps.executeQuery();
```

Step 6 – Close Resources

```
rs.close();  
ps.close();  
con.close();
```

5. Complete JDBC Program

ProductDAO.java

```
import java.sql.*;  
public class ProductDAO {  
    private static final String URL =  
        "jdbc:mysql://localhost:3306/ecommerce";  
    private static final String USER = "root";  
    private static final String PASSWORD = "1234";  
    public void insertProduct(String name,  
        double price,  
        int quantity) {  
  
        String sql =  
            "INSERT INTO products(name, price, quantity) " +  
            "VALUES (?, ?, ?)";  
  
        try {  
            Class.forName("com.mysql.cj.jdbc.Driver");  
            Connection con =  
                DriverManager.getConnection(  
                    URL, USER, PASSWORD);  
            PreparedStatement ps =
```

```

        con.prepareStatement(sql);

        ps.setString(1, name);
        ps.setDouble(2, price);
        ps.setInt(3, quantity);
        int rows = ps.executeUpdate();

        if (rows > 0) {
            System.out.println(
                "Product inserted successfully.");
        }
        ps.close();
        con.close();
    } catch (ClassNotFoundException e) {
        System.out.println("JDBC Driver not found.");
        e.printStackTrace();
    } catch (SQLException e) {
        System.out.println("Database error.");
        e.printStackTrace();
    }
}

public void getProducts() {
    String sql =
        "SELECT id, name, price, quantity FROM products";

    try {

        Class.forName("com.mysql.cj.jdbc.Driver");

        Connection con =
            DriverManager.getConnection(
                URL, USER, PASSWORD);

        Statement st = con.createStatement();

        ResultSet rs = st.executeQuery(sql);

        System.out.println("Product Details");
        System.out.println("-----");

        while (rs.next()) {

            int id = rs.getInt("id");
            String name = rs.getString("name");
            double price = rs.getDouble("price");
            int quantity = rs.getInt("quantity");

            System.out.println(
                id + " | " +
                name + " | " +
                price + " | " +
                quantity
            );
        }

        rs.close();
        st.close();
        con.close();

    } catch (ClassNotFoundException e) {

```

```

        System.out.println("JDBC Driver not found.");
        e.printStackTrace();

    } catch (SQLException e) {

        System.out.println("Database error.");
        e.printStackTrace();
    }
}

public static void main(String[] args) {

    ProductDAO product = new ProductDAO();

    // Insert product
    product.insertProduct(
        "Laptop",
        55000.00,
        10
    );

    // Retrieve products
    product.getProducts();
}
}

```

6. JDBC Exception Handling

JDBC programs can produce different errors such as:

- Database connection failure.
- Invalid SQL query.
- Incorrect database credentials.
- JDBC driver not found.
- Table or column not found.

These exceptions should be handled using try-catch.

```

try {

    Connection con =
        DriverManager.getConnection(URL, USER, PASSWORD);

    // Database operations

} catch (SQLException e) {

    System.out.println("Database error occurred.");
    e.printStackTrace();

} finally {

    // Close resources if necessary
}

```

Using PreparedStatement is also recommended because it helps prevent **SQL injection** and makes parameterized SQL queries easier to manage.

JDBC Workflow

Start

|

Load JDBC Driver

|

Establish Database Connection

|
vCreate Statement / PreparedStatement
|
Execute SQL Query
|
Process ResultSet
|
Close ResultSet
|
Close Statement
|
Close Connection
|
End

Overall Conclusion

Servlets provide the main server-side processing mechanism for Java web applications. **GET and POST** methods allow the application to receive client requests, while the servlet life cycle manages servlet creation, request processing, and destruction. **Http Session and Cookies** provide session tracking so that users can remain logged in across multiple pages. **JDBC** connects Java web applications with databases and allows applications such as e-commerce systems to insert, retrieve, update, and manage product information efficiently.
